Solución Taller — Pilas, Colas y Listas Enlazadas

Asignatura: Estructura de Datos — Institución Universitaria Pascual Bravo
Estudiante: Kevin Andres Correa

Parte 1 — Reconocer la estructura adecuada

1. Un editor de texto guarda las acciones recientes para permitir deshacer primero la última acción realizada:

Estructura: Pila (Stack).
Explicación: Esto es un comportamiento puramente LIFO (Last In, First Out). Hagan de cuenta que es un historial donde la última instrucción que se registra queda arriba de todo; al presionar deshacer, lo primero que se vuela es ese último cambio para regresar al estado anterior.

2. Las personas esperan su turno en una fila de atención y se atiende primero a quien llegó primero:

Estructura: Cola (Queue).
Explicación: Sigue la lógica de una estructura FIFO (First In, First Out). El primer elemento que ingresa a la fila de espera es obligatoriamente el primero en ser procesado y despachado, tal cual pasa en un sistema de turnos convencional.

3. Una aplicación recorre las canciones de una lista y, al terminar la última, vuelve automáticamente a la primera:

Estructura: Lista circular.
Explicación: En esta variante de lista, el puntero "siguiente" del último nodo no apunta a null, sino que se enlaza directamente con la dirección de memoria del nodo cabeza. Eso genera un bucle infinito ideal para ciclos continuos de reproducción.

4. Un sistema necesita recorrer una colección de registros en un único sentido y agregar elementos al comienzo con frecuencia:

Estructura: Lista enlazada simple.
Explicación: Como solo se requiere iterar en una sola dirección, basta con una lista simple. Insertar en la cabeza es supereficiente (O(1)) porque solo creamos el nuevo nodo, hacemos que apunte a la cabeza actual y reasignamos el puntero de inicio, sin necesidad de desplazar el resto de elementos en memoria.

5. Un reproductor permite pasar a la canción siguiente y a la anterior desde cualquier canción:

Estructura: Lista doblemente enlazada.
Explicación: Al implementar dos referencias por nodo (un puntero al nodo siguiente y otro al nodo anterior), podemos movernos de manera bidireccional a lo largo de toda la secuencia sin importar en qué posición estemos parados.

Parte 2 — Pilas (LIFO)

2.1 Sigue las operaciones

Secuencia de estados:
1. push(4) → Pila: [4] (Tope: 4)
2. push(9) → Pila: [4, 9] (Tope: 9)
3. push(2) → Pila: [4, 9, 2] (Tope: 2)
4. pop() → Retorna 2. Pila: [4, 9] (Tope: 9)
5. push(7) → Pila: [4, 9, 7] (Tope: 7)
6. peek() → Consulta el tope. Retorna 7. Pila: [4, 9, 7] (Tope: 7)
7. pop() → Retorna 7. Pila: [4, 9] (Tope: 9)
8. pop() → Retorna 9. Pila: [4] (Tope: 4)

Elemento final en la estructura: 4
Tope final: 4

Paso	1	2	3	4	5	6	7	8
Operación	push(4)	push(9)	push(2)	pop()	push(7)	peek()	pop()	pop()
Contenido (T→B)	[4]	[9,4]	[2,9,4]	[9,4]	[7,9,4]	[7,9,4]	[9,4]	[4]
Tope Actual	4	9	2	9	7	7	9	4
Resultado	-	-	-	devuelve 2	-	muestra 7	devuelve 7	devuelve 9

2.2 Casos límite

1. **pop() sobre pila vacía:** Desencadena una excepción por desbordamiento negativo o *underflow*. Para prevenir que colapse, metemos un condicional que valide el estado de la pila; si está vacía, lanzamos una alerta o retornamos un valor nulo.

2. **peek() sobre pila vacía:** Es el mismo escenario; no existe un nodo en el tope para inspeccionar. Lo controlamos verificando antes el estado y retornando un indicador de vacío (como *null*).

3. **Pseudocódigo para deshacer(pila):**

```
funcion deshacer(pila)
    si pila.estaVacia() entonces
        escribir "Error: No existen acciones en el historial."
        devolver nulo
    sino
        ultimaAccion = pila.pop()
        devolver ultimaAccion
    fin si
fin funcion
```

2.3 Aplicación: deshacer acciones

```
Pila inicial:
| pegar | <- Tope (Último evento)
```

```
| borrar |
| escribir | <- Base
```

Primer deshacer: Remueve *pegar*. Queda *borrar* en el tope.
Segundo deshacer: Remueve *borrar*. Queda *escribir* en el tope y base.
¿Qué acción se deshace primero? Se ejecuta primero el deshacer sobre **pegar**.
Explicación: Al ser una estructura lineal LIFO, el último comando registrado queda en la cima del stack. Cuando se invoca el método para revertir cambios, el sistema extrae directamente lo que está apuntado por el tope, que coincide con la acción más reciente.

Parte 3 — Colas (FIFO)

3.1 Simula la fila de atención

Flujo paso a paso:
1. encolar("Ana") → Cola: [Ana]
2. encolar("Luis") → Cola: [Ana, Luis]
3. encolar("Sol") → Cola: [Ana, Luis, Sol]
4. desencolar() → Devuelve a Ana. Cola: [Luis, Sol]
5. encolar("Mia") → Cola: [Luis, Sol, Mia]
6. desencolar() → Devuelve a Luis. Cola: [Sol, Mia]
7. encolar("Teo") → Cola: [Sol, Mia, Teo]

Resultados de las salidas: El primer desencolar() procesa a Ana y el segundo a Luis.
Nodos remanentes en espera: Quedan en memoria Sol, Mia y Teo en ese orden estricto (Sol en el frente y Teo en el final).

3.2 Operaciones y casos límite

1. **Pseudocódigo para atender(cola):**

```
funcion atender(cola)
    si cola.estaVacia() entonces
        escribir "Cola vacía: No hay elementos para procesar."
        devolver nulo
    sino
        elementoProcesado = cola.desencolar()
        devolver elementoProcesado
    fin si
fin funcion
```

2. **Manejo de desencolar en cola vacía:** Si se intenta sacar un elemento cuando está vacía, se produce una falla en tiempo de ejecución. Evaluamos previamente con el método booleano estaVacia() antes de actualizar las referencias.

3. **¿Por qué agregar al frente y retirar del final NO altera el comportamiento FIFO si se mantienen esas operaciones?** Porque aunque se intercambien mecánicamente los nombres de las operaciones, la lógica de prioridad se mantiene intacta: los elementos que llevan más tiempo guardados seguirán saliendo primero por el extremo de salida. Sigue siendo FIFO, solo que invertimos la dirección del flujo.

Parte 4 — Listas enlazadas simples

4.1 Construye y modifica una lista

```
1. Insertar 18 al final: Cabeza → [ 18 | sig ] → null
2. Insertar 7 al inicio: Cabeza → [ 7 | sig ] → [ 18 | sig ] → null
3. Insertar 24 al final: Cabeza → [ 7 | sig ] → [ 18 | sig ] → [ 24 | sig ]
→ null
4. Insertar 13 después de 7: Cabeza → [ 7 | sig ] → [ 13 | sig ] → [ 18 |
sig ] → [ 24 | sig ] → null
5. Eliminar el nodo 18: Cabeza → [ 7 | sig ] → [ 13 | sig ] → [ 24 | sig ] →
null

Diagrama final: Cabeza → [ 7 | sig ] → [ 13 | sig ] → [ 24 | sig ] → null
```

4.2 Diseña operaciones

1. **Recorrer la lista e imprimir todos sus datos:**

```
funcion recorrerEImprimir(cabeza)
    auxiliar = cabeza
    mientras auxiliar != nulo hacer
        escribir auxiliar.dato
        auxiliar = auxiliar.siguiente
    fin mientras
fin funcion
```

2. **Buscar un valor y devolver si está presente:**

```
funcion buscar(cabeza, valorBuscado)
    auxiliar = cabeza
    mientras auxiliar != nulo hacer
        si auxiliar.dato == valorBuscado entonces
            devolver Verdadero
        fin si
        auxiliar = auxiliar.siguiente
    fin mientras
    devolver Falso
fin funcion
```

3. **Insertar un nodo al comienzo:**

```
funcion insertarAlComienzo(cabeza, nuevoValor)
    nuevoNodo = crearNodo(nuevoValor)
    nuevoNodo.siguiente = cabeza
    cabeza = nuevoNodo
    devolver cabeza
fin funcion
```

4.3 Comprueba los bordes

* **Si la lista está vacía:** cabeza apunta a null. El algoritmo asigna nuevoNodo.siguiente = null y luego actualiza cabeza para apuntar a nuevoNodo. Se resuelve correctamente.
* **Si la lista ya contiene elementos:** El nuevoNodo.siguiente copia la dirección que tiene cabeza (enlazando al antiguo primer elemento). Inmediatamente, cabeza pasa a apuntar al nuevoNodo, manteniendo la integridad.

* **Referencias al eliminar el primer nodo:** Solo se reasigna la variable cabeza para que tome la dirección del segundo elemento (cabeza = cabeza.siguiente). El recolector de basura se encarga del nodo huérfano.

Parte 5 — Listas circulares

5.1 Reproduce un recorrido circular

Lista circular inicial: [P1 | sig] → [P2 | sig] → [P3 | sig] --(vuelve a P1)-->

Registro de 7 turnos (ráfagas de tiempo):
• Turno 1: Procesa P1 → Siguiente: P2
• Turno 2: Procesa P2 (Ráfaga 1 de P2) → Siguiente: P3
• Turno 3: Procesa P3 → Siguiente: P1
• Turno 4: Procesa P1 → Siguiente: P2
• Turno 5: Procesa P2 (Ráfaga 2 de P2 → termina proceso y se remueve) → Siguiente: P3
• Turno 6: Procesa P3 → Siguiente: P1
• Turno 7: Procesa P1 → Siguiente: P3

Estado tras remover P2: [P1 | sig] → [P3 | sig] --(vuelve a P1)-->

Condición de parada: Almacenamos el puntero inicial (nodoInicio = actual). La condición del ciclo será mientras (actual != nodoInicio). Cuando el puntero vuelva a igualar la dirección de memoria guardada, sabremos que completamos una revolución completa.

5.2 Casos especiales

1. **Lista vacía:** cabeza es null, no hay enlaces por actualizar.
2. **Lista con un solo nodo:** El único nodo se apunta a sí mismo (nodo.siguiente = nodo). Al removerlo, cabeza pasa a ser null.
3. **Eliminar el primer nodo:** La referencia cabeza se desplaza al segundo nodo. Estamos obligados a recorrer la estructura hasta el último nodo para actualizar su puntero siguiente, garantizando que enlace a la nueva cabeza del ciclo.
4. **Eliminar el último nodo:** Se debe iterar hasta el penúltimo nodo y reconfigurar su campo siguiente para que apunte hacia la cabeza, cerrando el ciclo de forma correcta.

Parte 6 — Listas doblemente enlazadas

6.1 Recorrido en ambos sentidos

```
Esquema de la lista doble:
null <- [ant | Lunes | sig] <=> [ant | Martes | sig] <=> [ant | Miércoles |
sig] <=> [ant | Jueves | sig] → null
```

Navegación Forward: Lunes → Martes → Miércoles → Jueves
Navegación Backward: Jueves → Miércoles → Martes → Lunes

6.2 Inserta y elimina

1. Insertar Domingo antes de Lunes: null <- [Domingo] <=> [Lunes] <=> [Martes] <=> ... → null
2. Insertar Viernes después de Jueves: null <- ... <=> [Jueves] <=> [Viernes] → null
3. Eliminar Miércoles: null <- [Domingo] <=> [Lunes] <=> [Martes] <=================> [Jueves] <=> [Viernes] → null

Mutación de punteros en la eliminación de Miércoles:
• El puntero siguiente del nodo Martes ahora se enlaza a Jueves.
• El puntero anterior del nodo Jueves ahora apunta de vuelta a Martes.

6.3 Compara los enlaces

Ventaja competitiva: Permite una lectura bidireccional de manera nativa sin la necesidad de emplear algoritmos recursivos o estructuras auxiliares de ordenamiento.
Desventaja / Trade-off: Incrementa la sobrecarga de memoria (el overhead aumenta al tener que almacenar dos direcciones de memoria por cada nodo, ant y sig) y añade complejidad algorítmica, ya que cada inserción o eliminación duplica la cantidad de asignaciones de punteros requeridas.

Parte 7 — Análisis y elección

Estructura y representación	Operación	Complejidad	Justificación breve
Pila enlazada con referencia al tope	Insertar en el tope	O(1)	Reasigna el puntero del nuevo nodo al tope actual en tiempo co
Pila enlazada con referencia al tope	Retirar del tope	O(1)	Se remueve directamente el nodo apuntado por la cima y se mu
Cola enlazada con referencias al frente y final	Encolar al final	O(1)	La referencia directa al nodo final evita recorrer la estructura, en
Cola enlazada con referencias al frente y final	Desencolar del frente	O(1)	Se extrae la cabeza de la cola y se actualiza el puntero del frent
Lista simple sin referencia al último nodo	Insertar al comienzo	O(1)	Operación local directa en la cabeza; el resto de la estructura pe
Lista simple sin referencia al último nodo	Insertar al final	O(n)	Al carecer de un puntero directo al final, es obligatorio iterar a tra
Lista doble con referencia a un nodo cualquiera	Desconectar ese nodo	O(1)	Usando los punteros anterior y siguiente del nodo actual, se enla

Preguntas adicionales

1. **¿Por qué una cola enlazada que solo guarda el frente tarda más en encolar al final?** Al no contar con una referencia directa al último nodo (tail), el hilo de ejecución se ve forzado a iniciar desde el frente (head) y avanzar secuencialmente a través de todos los nodos hasta encontrar el puntero que apunta a null. Esto eleva el costo computacional a un orden lineal O(n).

2. **¿Qué ventaja ofrece la lista circular para volver repetidamente al primer elemento?** Elimina la necesidad de implementar condicionales de control para evaluar si el puntero llegó a null. Al estar el último elemento linkeado nativamente con la cabeza, el ciclo se mantiene de manera fluida y limpia a nivel de código.

3. **¿Qué información adicional almacena una lista doble frente a una simple y qué permite hacer?** Guarda un puntero adicional por nodo dedicado a la dirección de memoria previa (anterior). Esto nos habilita para realizar búsquedas e iteraciones en reversa, además de poder remover cualquier nodo intermedio conocido en O(1) sin tener que hacer recorridos previos para averiguar cuál

era su nodo predecesor.

Parte 8 — Reflexión

A nivel de ingeniería de software, la forma en que estructuramos y conectamos los datos en memoria afecta de manera directa el rendimiento y la eficiencia de los algoritmos. Una pila es excelente para un flujo estrictamente inverso (LIFO), ideal para resolver mecanismos de reversión de estados o el manejo del call stack en ejecución. Por otro lado, las colas modelan sistemas de colas de procesos y mensajería sincrónica bajo principios equitativos (FIFO).

Cuando pasamos a analizar listas, la elección depende enteramente del patrón de acceso: una lista simple ofrece un diseño optimizado si solo necesitamos movernos hacia adelante y priorizamos un bajo consumo de memoria; el enfoque circular optimiza los planificadores de tareas de los sistemas operativos (como el Round Robin); y las listas dobles, a pesar de penalizar ligeramente el consumo de memoria por el almacenamiento de punteros redundantes, otorgan flexibilidad total para la navegación bidireccional y la mutación de nodos en tiempo constante. Al final, no existe una estructura intrínsecamente superior, sino soluciones optimizadas para requerimientos técnicos específicos.